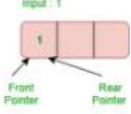
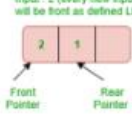
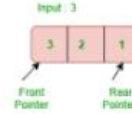
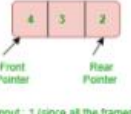
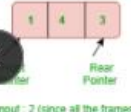
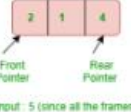


QUEUE

Q8) LRU Cache Implementation using Queue

DS	Session	Queue	Question Information	Level 1	Challenge 58
Question description Sekon is very interested in surfing the contents from google. He searches for various coding test on Google. One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the google suggestion list. first question for the coding competition is LRU cache implementation using queue concepts. Function Description 1. Queue which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end. 2. A Hash with page number as key and address of the corresponding queue node as value.					
Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.  Input : 1  Input : 2 (every new input will be front as defined LRU)  Input : 3  Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)  Constraints For this experiment, cache can hold 4 pages. Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9). Input Format: First line represents n and m, where n is the page number with in the range [0-9] & m is cache size (must be 4 for this problem). Next line represents the reference pages. Output Format: Single line represents the cache frames after the above referenced pages.					

CODE:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct QNode { struct QNode *prev, *next; unsigned pageNumber; } QNode;
5  typedef struct Queue { unsigned count, numberOfFrames; QNode *front, *rear; } Queue;
6
7  QNode* newQNode(unsigned pageNumber) {
8      QNode* t = (QNode*)calloc(nmemb:1, size: sizeof(QNode));
9      return t->pageNumber = pageNumber, t;
10 }
11
12 Queue* createQueue(int numberOfFrames) {
13     Queue* q = (Queue*)calloc(nmemb:1, size: sizeof(Queue));
14     return q->numberOfFrames = numberOfFrames, q;
15 }
16
17 void detach(Queue *q, QNode *c) {
18     if (c->prev) c->prev->next = c->next; else q->front = c->next;
19     if (c->next) c->next->prev = c->prev; else q->rear = c->prev;
20 }
21
22 void pushFront(Queue *q, QNode *c) {
23     c->prev = NULL; c->next = q->front;
24     if (q->front) q->front->prev = c; else q->rear = c;
25     q->front = c;
26 }
27
28 int main(void) {
29     int n, m, p;
30     if (scanf(format: "%d%d", &n, &m) != 2) return 0;
31     Queue* q = createQueue(numberOfFrames: m);
32     QNode* map[100] = {0};
33
34     while (n-- && scanf(format: "%d", &p)) {
35         if (map[p]) detach(q, c: map[p]);
36         else {
37             if (q->count == q->numberOfFrames) {
38                 map[q->rear->pageNumber] = NULL;
39                 QNode *old = q->rear;
40                 detach(q, c: old); free(ptr: old);
41             } else q->count++;
42             map[p] = newQNode(pageNumber: p);
43         }
44         pushFront(q, c: map[p]);
45     }
46
47     for (QNode* c = q->front; c; c = c->next) printf(format: "%d%s", c->pageNumber, c->next ? " " : "");
48     return printf(format: "\n"), 0;
49 }

```

OUTPUT:

```

[suraj@suraj-83gs test]$ ./main
6 4
1 2 3 1 4 5
5 4 1 3
[suraj@suraj-83gs test]$

```